After Acceptance: A Claim Graph for Residual-Machine Claims, Calibrated on eBPF

Chengao Zhang
Independent Researcher
emtanling@gmail.com

***Abstract*—Language-theoretic security asks whether a boundary recognizes the language that downstream machinery actually interprets. At a program-verifier boundary, an accepted artifact may drive stateful runtime operations not described by an intended report relation. We introduce a five-node claim graph separating artifact acceptance (A), a same-suffix causal state distinction (C), bounded programmability (P), output-witnessed computed-report non-factorization (R), and a policy/threat obligation (W). A future-observation quotient and behavioral factorization criterion distinguish the accepted-artifact-indexed causal language from the report-relative residual language. The criterion is report-instance-relative, applies only under an explicit unique-cell report map on the chosen context fiber, and does not assert that every safety-sound incomplete verifier hosts a weird machine.**

We give proof obligations for a uniformly controlled, observable, resettable gate and one fixed accepted interpreter over a bounded circuit-description domain. Prefix induction yields bounded composition under exact scheduling, admissibility, serialization, and frame-preservation premises. In a Linux eBPF calibration, a dedicated preallocated non-LRU two-entry hash map realizes NAND: reset leaves one sentinel entry; input bits select existing- or fresh-key updates; and the second update's success predicate supplies the output. One fixed accepted program consumes canonical NAND DAGs with at most 64 inputs, 512 gates, and 578 live wires.

A source-snapshotted Linux/aarch64 run covers named and random circuits, joint bounds, serial reuse, mechanism controls, and malformed descriptors. A separate author-run semantic auditor reconstructs descriptors and circuit semantics, and a self-issued manifest checks bundle integrity. The case records A, gives a conditional C witness under the declared concrete-service/no-interference contract, and supports P under additional implementation premises. It does not establish Linux report non-factorization or a policy-level weird machine. Separately, a fixed auxiliary executable instance establishes $R(M_{linux\_r\_aux\_v1})$ for its own computed report; it does not change the stock-Linux result.

***Index Terms*—language-theoretic security, recognizers, residual languages, weird machines, program verification, abstract interpretation, eBPF**

I. Introduction

Language-theoretic security treats an input boundary as a language-recognition boundary: downstream processing is interpretation, and the input supplies the interpreted program [1]–[4]. We use the same recognizer-shaped model for program verification. Our question begins after acceptance. A verifier accepts a program artifact, but the accepted program can still drive a language of stateful helper and service operations. Which claims are needed before that downstream language can be called residual, programmable, shape-induced, or a weird machine?

The artifact language and the post-acceptance operation language have different carriers. A verifier can recognize bounded execution, typed helper use, and memory-access discipline without certifying the complete relation implemented through every documented runtime service. The accepted program may select operations, retain service state, observe returns, reset state, and compose effects. None of this alone implies a verifier defect. It first establishes a downstream operation language indexed by an accepted artifact.

The paper organizes the evidentiary burden as a claim graph:

The A, C, P, and W status cells in Table I report the stock eBPF/Linux case. The R cell separates two carriers: $M_{linux\_r\_aux\_v1}$ establishes R for the auxiliary report-producing recognizer and restricted service semantics, while R for the stock Linux verifier remains unestablished.

Nodes C and R must not share one name. We call the accepted-artifact-indexed, same-suffix language L_{causal}. A word enters the report-relative residual language L^R_{res} only with an actual computed-cell collision under the admissibility conditions below. P and R branch after C. A policy-level weird machine requires P, W, linkage to the same encoded computation, and unintendedness; a *contract-shape-induced recognizer-relative* classification additionally requires R, documented semantics, report conformance, and granularity evidence. The graph turns "weird machine" from a rhetorical label into explicit, auditable obligations.

Programmability is a separate constructive problem. A state distinction may be dead, uncontrolled, one-shot, or impossible to compose. We therefore require a program-visible readout, one uniform input dispatcher, a reset to a canonical class, and one fixed accepted interpreter with exact scheduling and frame preservation over an independently declared bounded descriptor domain. The resulting composition theorem is a proof-obligation decomposition, not a universal necessity theorem.

Our calibration case is Linux eBPF. A fixed artifact uses a dedicated two-entry hash map as a saturating resource. After reset, one sentinel is live. A zero bit updates the sentinel; a one bit inserts a fresh input-specific key. The second input-conditioned update succeeds except on input $(1, 1)$, yielding NAND. Ordinary bytecode still validates descriptors, selects keys, routes wires, controls the loop, and stores outputs. A host parser normalizes textual WMC1 into maps, while the verifier accepts only the fixed eBPF artifact.

TABLE I
CLAIM GRAPH AND EVIDENCE STATUS.

Node	Required fact	Evidence status
A	**Acceptance:** one fixed artifact belongs to L_V	recorded for the preserved object and environment
C	**Causal state distinction:** the same suffix and non-state context expose different observations from different selected runtime states	conditional witness under the declared map-service/no-interference contract
P	**Bounded programmability:** accepted code uniformly controls, observes, resets, and composes the state under explicit frame and environment premises	source-level construction plus audited regression evidence
R	**Report-relative residual:** witnesses at C are jointly covered by an actual computed report cell that fails the behavioral factorization test	auxiliary fixed tuple: established; stock Linux: not established
W	**Policy/threat obligation:** an actor drives an effect through P_* that the policy excludes	not established by the offline case

The case is deliberately diagnostic. It records A, gives a conditional C witness under the declared service contract, and supports P under stated implementation premises, even though ordinary bytecode easily expresses the same function. It also shows why P cannot be promoted to R or combined with an absent W obligation. The retained verifier log is not a computed-cell extractor, and the offline run supplies no violated policy.

The paper contributes:

1) **A claim graph and typed languages.** We distinguish accepted artifacts, causal runtime words, report-relative residual words, bounded programmability, and policy/threat obligations. A future-observation quotient and factorization criterion make node R testable, while countermodels show that the branches do not collapse.
2) **A bounded composition argument.** E1–E3 and the precise E4-D interpreter obligations isolate local gate correctness, reset, scheduling, and global frame conditions. Prefix induction proves the functional result; safety is a separate optional premise.
3) **An eBPF calibration case.** A saturating-rank NAND basis and fixed interpreter target $D_{64,512}$. The recorded suite covers named and fixed-seed random DAGs, deep and joint boundaries, a zero-gate case, serial reuse, mechanism controls, and malformed descriptors.
4) **An executable report instance.** A fixed auxiliary recognizer/service tuple emits an actual report, constructs the finite future-observation quotient, and independently checks a report-cell collision and four negative controls. This establishes R only for that named tuple while preserving the stock-Linux boundary.

The empirical scope is one privileged, offline Linux/aarch64 environment. The case is not a Linux report-opacity result, concurrent deployment result, artifact-parametric compiler, unbounded machine, vulnerability, or policy-level weird machine.

II. RECOGNIZER-RELATIVE RESIDUAL LANGUAGES

A. Recognition, execution, and report

Let V be a recognizer over program artifacts and I the concrete execution semantics over state carrier Σ_I and trace carrier $\mathcal{T}_I$, including the instruction engine, helpers, stateful services, and relevant environment. Thus $\mathsf{Tr}_I(P) \subseteq \mathcal{T}_I$. The accepted artifact language is

$$L_V = \{P \mid V(P) = \mathsf{accept}\}. \tag{1}$$

For an optional set $\mathsf{Safe} \subseteq \mathcal{T}_I$ of permitted concrete traces, the boundary is safety-sound when

$$\forall P \in L_V.\ \mathsf{Tr}_I(P) \subseteq \mathsf{Safe}. \tag{2}$$

Safety soundness is a premise, not an inference from one successful load. It is also distinct from completeness of an abstract transformer. Abstract interpretation relates concrete sets and abstract elements through abstraction and concretization maps [5], but a verifier's accepted language, its computed report, transfer soundness, and completeness are different judgments. In particular, the best abstraction of a singleton need not be the analyzer-computed cell at a joined control frontier.

When a report is in scope, let $\mathsf{Report}_V(P, \ell) \subseteq A_\ell$ be the finite set of abstract cells actually computed at frontier ℓ, with a declared concretization $\gamma_\ell : A_\ell \to \mathcal{P}(\Sigma_I)$. Frontier coverage requires

$$\mathsf{Reach}_I(P, \ell) \subseteq \bigcup_{a^\# \in \mathsf{Report}_V(P,\ell)} \gamma_\ell(a^\#). \tag{3}$$

Two concrete states are jointly covered only when one computed cell contains both. This point rules out a common but invalid shortcut: showing that two values have similar printed log text does not identify a computed abstract cell, a concretization, or joint coverage.

B. Causal words

For $P \in L_V$ and frontier ℓ, let $\Sigma_{\mathrm{op}}(P)$ be a finite alphabet of program-controllable runtime operations, and let $W^\ell_{\mathrm{run}}(P) \subseteq \Sigma_{\mathrm{op}}(P)^*$ be the words accepted code can execute from ℓ. Calling every such word residual would rename ordinary trace semantics, so node C uses a same-suffix causal test without yet asserting report omission.

Fix before comparison an observation contract

$$K_{\mathrm{obs}} = (\rho_{\mathrm{obs}}, \mathsf{Obs}, \mathsf{Slice}, \mathsf{Env}). \tag{4}$$

Here $\rho_{\mathrm{obs}} : \Sigma_I \to R_{\mathrm{obs}}$ selects the candidate state, $\mathsf{Obs} : \mathcal{T}_I \to O_{\mathrm{obs}}$ projects a complete concrete trace, Slice assigns to each word w a typed context projection $\mathsf{ctx}_w : \Sigma_I \to C_w$, and

Env is the declared set of fixed environment instances. The context conservatively contains every non-selected component read by the suffix or observer; an environment instance fixes the relevant resource configuration, schedule, nondeterminism, and external-interference choice.

Write $[\![w]\!]_{I,e}(\sigma) = (\tau, \sigma')$ for a defined, terminating suffix execution in environment e. The contract is *sound for w on $X \subseteq \Sigma_I$* when, for every $e \in \mathsf{Env}$ and $\sigma, \sigma' \in X$ satisfying

$$\rho_{\mathrm{obs}}(\sigma) = \rho_{\mathrm{obs}}(\sigma') \quad \text{and} \quad \mathsf{ctx}_w(\sigma) = \mathsf{ctx}_w(\sigma'), \tag{5}$$

the two suffix executions have the same definedness, and, whenever $[\![w]\!]_{I,e}(\sigma) = (\tau, \sigma_1)$ and $[\![w]\!]_{I,e}(\sigma') = (\tau', \sigma_1')$, then $\mathsf{Obs}(\tau) = \mathsf{Obs}(\tau')$. Thus $(\rho_{\mathrm{obs}}, \mathsf{ctx}_w, e)$ contains every factor that may affect the declared observation. Without this noninterference condition no C witness is admitted.

Definition 1 (causal state-mediated word family). A word w is causal at (P, ℓ) when $P \in L_V$, $w \in W^{\ell}_{\mathrm{run}}(P)$, K_{obs} is sound for w on $\mathsf{Reach}_I(P, \ell)$, and there are $e \in \mathsf{Env}$ and $\sigma_0, \sigma_1 \in \mathsf{Reach}_I(P, \ell)$ for which $[\![w]\!]_{I,e}(\sigma_i) = (\tau_i, \sigma_i')$ are both defined and terminating, and

$$\begin{aligned} \mathsf{ctx}_w(\sigma_0) &= \mathsf{ctx}_w(\sigma_1), \\ \rho_{\mathrm{obs}}(\sigma_0) &\neq \rho_{\mathrm{obs}}(\sigma_1), \\ \mathsf{Obs}(\tau_0) &\neq \mathsf{Obs}(\tau_1). \end{aligned} \tag{6}$$

Define the dependent tagged family

$$L_{\mathrm{causal}}(V, I; K_{\mathrm{obs}}) = \{(P, \ell, w) \mid w \text{ is causal at } (P, \ell)\}. \tag{7}$$

The artifact and frontier tags are part of the type. With fixed, decodable prefix encodings for artifacts, frontiers, and operation symbols, the family can be embedded in an ordinary language over one finite alphabet; the dependent notation keeps the carriers visible. A host descriptor is neither another element of L_V nor automatically a runtime word. It first becomes configuration, the accepted interpreter induces a schedule, and only a same-suffix witness enters L_{causal}.

The definition is observer- and slice-relative. Deleting an earlier input from the context after seeing the result would make the test circular. The eBPF case therefore declares its suffix from source semantics before comparison; the translated dump is retained for manual inspection rather than consumed by an automated slice checker.

C. Behavioral quotient and report-relative residual language

Fix a deterministic discipline

$$D = (X_D, S_D, A_D, O_D, \delta_D, \lambda_D, s_D, \mathsf{Obs}_D) \tag{8}$$

with an admissible concrete region $X_D \subseteq \Sigma_I$, state carrier S_D, operation alphabet A_D, output alphabet O_D, observer $\mathsf{Obs}_D : O_D^* \rightharpoonup O_{\mathrm{obs}}$, and same-domain partial functions

$$\delta_D : S_D \times A_D \rightharpoonup S_D, \qquad \lambda_D : S_D \times A_D \rightharpoonup O_D. \tag{9}$$

Whenever D is used at (P, ℓ), fix an injective operation encoding $\iota_{P,\ell} : A_D \to \Sigma_{\mathrm{op}}(P)$ and extend it homomorphically. The pair $(s_D, \iota_{P,\ell})$ is *operationally adequate on X_D* when, for every $\sigma \in X_D$ and $a \in A_D$, concrete execution of $\iota_{P,\ell}(a)$ exists exactly when $\delta_D(s_D(\sigma), a)$ is defined; whenever $\sigma \xrightarrow{\iota_{P,\ell}(a)/o}_I \sigma'$, where $o \in O_D$ is the complete declared observation of that encoded operation, then $\sigma' \in X_D$ and

$$\lambda_D(s_D(\sigma), a) = o, \qquad s_D(\sigma') = \delta_D(s_D(\sigma), a). \tag{10}$$

Thus, within X_D, the encoding is a semantic renaming rather than an arbitrary map: one projected state cannot hide different definedness, outputs, or successor projections. Every environment choice that affects a transition is fixed by D or included in S_D.

Let $\mathsf{Out}_D(r, w)$ be the partial complete output word obtained by iterating (δ_D, λ_D), write $\mathsf{Def}_D(r, w)$ for $\mathsf{Out}_D(r, w) \downarrow$, and take the continuation universe to be $\mathcal{W}_D = A_D^*$; infeasible words are represented by undefined output. For $r, r' \in S_D$, define

$$\begin{aligned} r \sim_D r' \quad \Longleftrightarrow \quad & \forall w \in \mathcal{W}_D. \\ & \Big[\mathsf{Def}_D(r, w) \Longleftrightarrow \mathsf{Def}_D(r', w)\Big] \\ \wedge \, & \Big[\mathsf{Def}_D(r, w) \Rightarrow \mathsf{Out}_D(r, w) = \mathsf{Out}_D(r', w)\Big]. \end{aligned} \tag{11}$$

Because $\mathcal{W}_D = A_D^*$ is closed under left-prefixing by an operation, $\sim_D$ is a right congruence: defined matching a-steps lead to equivalent successor states, since every successor continuation w is tested as aw. This follows the classical continuation-equivalence lineage of Nerode and Mealy machines [6], [7]; the partial-output setting and observer contract here are paper-specific. If $Q_D = S_D/\!\sim_D$ is finite, execution restricted to X_D induces a partial Mealy transducer over quotient states. Observational completeness and generalized strong preservation likewise ask whether an abstraction preserves a selected observation or specification language [8], [9]. Our test is narrower: it concerns cells actually computed for one accepted artifact at one frontier. It is a semantic quotient test, not a claim to introduce a new general completeness theory.

For a concrete set F, define the common enabled continuations

$$\mathcal{W}_D(F) = \{w \in \mathcal{W}_D \mid \forall \sigma \in F.\ \mathsf{Out}_D(s_D(\sigma), w) \downarrow\}. \tag{12}$$

Proposition 1 (behavioral factorization on a context fiber). Fix accepted P, frontier ℓ, discipline D, and observation contract K_{obs}. Let $\varnothing \neq F \subseteq \mathsf{Reach}_I(P, \ell) \cap X_D$. Fix an injective operation encoding $\iota_{P,\ell} : A_D \to \Sigma_{\mathrm{op}}(P)$ and its homomorphic extension to words. Assume

$$\iota_{P,\ell}(\mathcal{W}_D(F)) \subseteq W^{\ell}_{\mathrm{run}}(P), \tag{13}$$

that K_{obs} is sound on F for every encoded word $\iota_{P,\ell}(w)$ with $w \in \mathcal{W}_D(F)$, and that

$$\forall w \in \mathcal{W}_D(F).\ \forall \sigma, \sigma' \in F. \quad \mathsf{ctx}_{\iota_{P,\ell}(w)}(\sigma) = \mathsf{ctx}_{\iota_{P,\ell}(w)}(\sigma'). \tag{14}$$

Require observation compatibility: whenever encoded $w \in \mathcal{W}_D(F)$ executes from $\sigma \in F$ with concrete trace τ, the following right-hand side is defined and

$$\mathsf{Obs}(\tau) = \mathsf{Obs}_D(\mathsf{Out}_D(s_D(\sigma), w)). \tag{15}$$

Define

$$\beta_D(\sigma) = [s_D(\sigma)]_D, \qquad F_{a^\#} = F \cap \gamma_\ell(a^\#). \tag{16}$$

Finally, require the unique-cell condition

$$\forall \sigma \in F.\ \exists! a^\# \in \mathsf{Report}_V(P,\ell).\quad \sigma \in \gamma_\ell(a^\#), \tag{17}$$

and let $\pi_R : F \to \mathsf{Report}_V(P,\ell)$ map each state to that unique computed cell. Then the report factors the future-observation quotient on F,

$$\exists h : \pi_R(F) \to Q_D.\quad \beta_D = h \circ \pi_R, \tag{18}$$

if and only if

$$\forall a^\# \in \pi_R(F).\quad |\beta_D(F_{a^\#})| \le 1. \tag{19}$$

Consequently, report non-factorization on F is equivalent to a unique computed cell containing two states from different $\sim_D$ classes.

Proof. If $\beta_D = h \circ \pi_R$, states with the same unique report label have the same quotient class, giving the cardinality bound. Conversely, the bound makes $h(a^\#)$ the unique element of $\beta_D(F_{a^\#})$ for every $a^\# \in \pi_R(F)$; this is well-defined and yields the factorization. Negating either equivalent condition gives the collision statement. □

Write $\mathsf{Adm}(P,\ell,D,F;K_{\mathrm{obs}})$ when all of the following hold: $P \in L_V$; $(s_D, \iota_{P,\ell})$ is operationally adequate on X_D; $\varnothing \neq F \subseteq \mathsf{Reach}_I(P,\ell) \cap X_D$; the runtime-word inclusion, observer compatibility, sound observation contract, common-context condition, and unique-cell condition above all hold. The last condition makes the nonempty $F_{a^\#}$ a disjoint cover. Reports with overlapping labels require a separately declared membership-signature map and are outside this criterion.

Definition 2 (report-relative residual language). Fix an admissible tuple. A dependent tagged word $(P,\ell,D,F,a^\#,w)$ is *output-witnessed report-relative residual* when:

$$\begin{aligned}
&(P,\ell,\iota_{P,\ell}(w)) \in L_{\mathrm{causal}}(V,I;K_{\mathrm{obs}}), \qquad w \in \mathcal{W}_D(F),\\
&\sigma_0,\sigma_1 \in F \text{ are Definition 1 witnesses for } \iota_{P,\ell}(w),\\
&\pi_R(\sigma_0) = \pi_R(\sigma_1) = a^\#, \qquad \beta_D(\sigma_0) \neq \beta_D(\sigma_1).
\end{aligned} \tag{20}$$

$L^R_{\mathrm{res}}(V,I,\mathsf{Report};K_{\mathrm{obs}})$ is the union of these tagged words over tuples satisfying Adm. Operational adequacy and observer compatibility make different admitted concrete observations imply different quotient classes; the explicit inequality binds the definition to Proposition 1. The quotient also distinguishes definedness, so report non-factorization can arise without a jointly terminating output-distinguishing word. Accordingly, L^R_{res} is the output-witnessed subset of report-factorization failures, not a converse characterization of all failures. It is a frontier collision, not a claim that a complete whole-program report can never recover the final function.

Instantiating Definition 2 requires an extractor for actual computed cells, a declared concretization, a unique-cell partition on the chosen fiber (or a separately defined alternative report map), and the remaining admissibility premises. The retained Linux log does not provide these objects, so the eBPF case gives only a conditional L_{causal} witness under its declared service contract and does not establish L^R_{res}.

D. Shape, defect, and policy

A defect-induced gap uses behavior that violates the declared recognizer, report, or runtime contract. A documented, safety-preserving witness in L^R_{res} for a relation the report intends to certify is evidence for a contract-shape gap, not by itself a shape proof: report conformance and granularity must rule out a faulty transformer, join, or extractor. A policy-level weird machine additionally needs linked actor control, a policy-excluded effect, and unintended interpretation. An ordinary stateful API can therefore implement a transducer without satisfying either classification.

III. Bounded State-Mediated Circuit Realization

A. From a distinction to a reusable gate

A causal word can expose one distinction without supporting repeated computation. Fix a deterministic gate discipline D, one canonical reset class $q_0 \in Q_D$ identified with its subset of S_D, and a nonempty admissible set $\mathsf{Adm}_G \subseteq X_D$. A gate basis $(P_G, \mathit{reset}, G, \mathit{observe}, D, \mathsf{Adm}_G)$ uses one accepted artifact P_G, a dispatcher G selecting a complete word $u_G(x) \in A_D^*$ for each $x \in \{0,1\}^2$, and a partial readout $\mathit{observe} : O_D^* \rightharpoonup \{0,1\}$. For this basis, $O_{\mathrm{obs}} = \{0,1\}$ and $\mathsf{Obs}_D = \mathit{observe}$. Its predeclared operation encoding into $\Sigma_{\mathrm{op}}(P_G)$ is operationally adequate on X_D. The basis satisfies:

E1 — causal basis and observation. There exist inputs x_0, x_1, prefixes $p_0, p_1 \in A_D^*$, one common remaining suffix $w \in A_D^*$, reset-class states $\eta_0, \eta_1 \in \mathsf{Adm}_G$, and one internal frontier ℓ_G such that

$$u_G(x_i) = p_i w, \qquad s_D(\eta_i) \in q_0. \tag{21}$$

Executing each p_i reaches a state σ_i at ℓ_G; the two σ_i are Definition 1 witnesses for the encoded common suffix w. If its traces are τ_i, then the complete gate readout is precisely that causal observation:

$$\mathit{observe}(\mathsf{Out}_D(s_D(\eta_i), u_G(x_i))) = \mathsf{Obs}(\tau_i), \qquad i \in \{0,1\}. \tag{22}$$

Accepted code uses this readout as the gate result that E4-D writes; it is not an unrelated probe.

E2 — uniform input control. The dispatcher G in P_G reads runtime bits $x \in \{0,1\}^2$. For every $\sigma \in \mathsf{Adm}_G$ with $s_D(\sigma) \in q_0$, $\mathsf{Out}_D(s_D(\sigma), u_G(x))$ and its readout are defined and

$$\mathit{observe}(\mathsf{Out}_D(s_D(\sigma), u_G(x))) = g(x) \tag{23}$$

for one fixed $g : \{0,1\}^2 \to \{0,1\}$. The experimenter is not an external oracle choosing a different constant program for each input.

E3 — reset. The reset word satisfies $\mathit{reset} \in A_D^*$. From every $\sigma \in \mathsf{Adm}_G$, it is defined, preserves declared wire cells, and terminates in $\tau \in \mathsf{Adm}_G$ with $s_D(\tau) \in q_0$. If two pre-states in Adm_G agree on those wire cells and the fixed context and reset to τ_0, τ_1, then for every $x \in \{0,1\}^2$,

$$\mathsf{Out}_D(s_D(\tau_0), u_G(x)) = \mathsf{Out}_D(s_D(\tau_1), u_G(x)). \tag{24}$$

Without E1, a hidden difference has no program-visible effect. Without E2, the experimenter may supply the truth table. Without E3, the channel may be one-shot. A fourth condition below composes the gate without placing circuit semantics in an external oracle.

B. Descriptor domain

The artifact uses the bounded domain $D_{64,512}$. A descriptor is

$$d = (m, n, (s_i^0, s_i^1)_{0 \le i < n}), \tag{25}$$

with $0 \le m \le 64$, $0 \le n \le 512$, and

$$0 \le s_i^b < 2 + m + i \tag{26}$$

for each gate i and operand b. Write $m(d) = m$ and $n(d) = n$. Wire 0 is constant zero, wire 1 is constant one, primary inputs occupy wires 2 through $2+m-1$, and gate i writes canonical destination $2 + m + i$. Thus the maximum number of live canonical wires is $2 + 64 + 512 = 578$ and the highest valid wire index is 577.

For $x \in \{0,1\}^{m(d)}$ and gate function g, $\mathsf{Eval}_g(d, x)$ initializes the constants and input vector, then extends the wire vector in descriptor order:

$$\nu[2 + m + i] = g(\nu[s_i^0], \nu[s_i^1]). \tag{27}$$

Its result is the complete canonical vector $(\nu[0], \ldots, \nu[1 + m(d) + n(d)])$, the same type later returned by $\mathsf{WireObs}_d$.

The host encoding $\mathsf{Enc}_U(d, x)$ writes a normalized descriptor, inputs, and control record into maps. Textual WMC1 is a host serialization. Neither WMC1 nor d is a BPF program accepted by V; $\mathsf{Sched}_U(d, x)$ is the operation schedule induced when accepted P_U reads the encoding.

For $c = \mathsf{Enc}_U(d, x)$, let $\mathsf{PhysRun}_U(c) = (s, k, \mu)$ contain final status, completed-iteration count, and physical map state. Define the descriptor-relative canonical observation

$$\mathsf{WireObs}_d(\mu) = (\mu[0], \ldots, \mu[1 + m(d) + n(d)]) \tag{28}$$

and the status-masked interface

$$\mathsf{Run}_{U,d}(c) = \begin{cases} (s, \mathsf{WireObs}_d(\mu)), & s = \mathsf{OK}, \\ (s, \bot), & s \ne \mathsf{OK}. \end{cases} \tag{29}$$

The host may project requested output wires only from an OK result. This semantic rule does not assert that stale physical cells are erased.

E4-D — bounded data-parametric interpretation. One fixed artifact P_U with a gate basis whose artifact component is the same P_U discharges E4-D for $D_{64,512}$ when:

1) for the fixed map definitions and recorded load environment, $P_U \in L_V$ independently of descriptor contents d and x;
2) every valid $c = \mathsf{Enc}_U(d, x)$ establishes a pre-callback state $\mu^{(0)}$ with $\mu^{(0)}[0] = 0$, $\mu^{(0)}[1] = 1$, and $\mu^{(0)}[2 + j] = x_j$ for $0 \le j < m(d)$, executes exactly callback positions $0, \ldots, n(d) - 1$ in order, and terminates with $\mathsf{PhysRun}_U(c) = (\mathsf{OK}, n(d), \mu)$;
3) one external critical section covers setup, invocation, and readback for every shared map in the footprint;
4) every iteration i validates canonical form and copies exactly its two earlier source wires; the complete concrete state σ_i immediately before reset belongs to Adm_G; the iteration then resets and invokes the E1–E3 gate, writes the resulting g bit only to destination $2 + m(d) + i$ and declared audit cells, and preserves the descriptor and all earlier wires; and
5) no execution performs more than 512 iterations, while malformed core-ABI controls or descriptor configurations covered by the declared validator terminate with non-OK status, at most 512 completed iterations, and semantic result $(s, \bot)$.

These clauses are deliberately proof obligations. In particular, E4-D requires the gate result to be written and exact iteration/terminal behavior; merely traversing a descriptor is insufficient.

C. Realization theorem

Theorem 1 (bounded composition under explicit obligations). Assume the fixed P_U discharges E4-D under its declared deterministic and serialized environment, and the gate basis on that same P_U satisfies E1–E3 with function g. Then, for every $d \in D_{64,512}$ and $x \in \{0,1\}^{m(d)}$,

$$\mathsf{Run}_{U,d}(\mathsf{Enc}_U(d, x)) = (\mathsf{OK}, \mathsf{Eval}_g(d, x)) \tag{30}$$

after exactly $n(d)$ iterations. If the recognition boundary is additionally safety-sound for Safe, these traces also satisfy Safe.

Proof sketch. E4-D initializes the constant/input prefix. At iteration i, the descriptor bound makes both sources earlier cells. E3 supplies q_0, E2 identifies the readout with g from that class, and E4-D writes that same readout only to the canonical destination while preserving the established prefix. Prefix induction yields Eval_g after $n(d)$ iterations; the exact-schedule clause supplies terminal OK. E1 binds the bit used by E2 and E4-D to a causal same-suffix state distinction; without E1 the functional induction would show only an ordinary bounded g-evaluator, not a state-mediated one. Safety follows only from the separate soundness premise. □

The theorem decomposes local gate, reset, scheduling, and frame obligations; it is not a claim that tests enumerate the descriptor domain. It also does not prove E4-A, in which a compiler emits a different accepted BPF object per circuit.

IV. eBPF Calibration Case

A. Execution boundary

The program $P_U = wm\_circuit$ is one fixed eBPF artifact with section $SEC(\text{syscall})$. In the recorded environment the in-kernel verifier accepts it and userspace executes it offline through $bpf\_prog\_test\_run\_opts()$; Linux documents both userspace program execution and the syscall section/program-type mapping [10], [11]. No live hook is involved. The experiment is privileged and local, and its acceptance and

resource facts remain specific to the recorded program type, kernel, and architecture.

The carrier boundary is:

The gate map $G0$ is a dedicated $BPF\_MAP\_TYPE\_HASH$ map with $max\_entries = 2$. It is non-LRU and retains the default preallocation; Linux documents the entry bound, default preallocation, update flags, and success/negative-error convention for this map type [12]. The discipline requires successful reset and one external critical section over setup, invocation, and readback for every map in the footprint. The serial harness satisfies the no-interleaving use pattern, but the eBPF program implements no concurrency lock.

B. Saturating-rank NAND

The gate uses pairwise-distinct keys: sentinel S and input keys A and B. Reset deletes all three and inserts S, establishing occupancy one. For the first input, zero updates S and one inserts fresh A; for the second, zero updates S and one inserts fresh B. Proposition 2 states the abstract occupancy law. For the Linux instance, existing-key success, below-capacity fresh-key success, and at-capacity fresh-key failure are explicit premises restricted to valid arguments, default preallocation, successful reset, no interference, and the recorded Linux 6.17 environment. Reference [12] supplies the map interface and return convention; retained raw returns and mechanism controls instantiate the law for this object. The gate observes only the second update's success predicate, not a portable error number.

The four executions are:

Proposition 2 (saturating-rank NAND). Let rank mean occupied-name count in a resource of capacity $k \geq 2$. Assume reset establishes rank $k-1$, S is among those occupied names, and pairwise-distinct A and B are fresh. Existing-name update succeeds without changing rank; fresh-name update succeeds and increments rank below k, and fails without changing rank at k. Dispatch zero to S, the first one to A, and the second one to B. The second update's success predicate is $\mathsf{NAND}(a, b)$.

Proof. When $a = 0$, the first update preserves rank $k-1$, so either second update succeeds. When $a = 1$, the first update raises the rank to k; the second update succeeds for $b = 0$ because S already exists and fails for $b = 1$ because B is fresh. The outputs are therefore $1, 1, 1, 0$. □

NAND is functionally complete because $\neg x = \mathsf{NAND}(x, x)$ and $x \wedge y = \neg\mathsf{NAND}(x, y)$; the 512-gate descriptor bound limits circuit size, not the Boolean basis.

For the concrete Definition 1 witness, take $P = wm\_circuit$ and let ℓ be the point in $circuit\_step\_cb$ immediately before the second map update, after the first input-conditioned operation. Let $R_{G0}(\sigma)$ be the complete helper-relevant dynamic state of the dedicated map $G0$—including key occupancy, buckets, preallocated element/free-list metadata, and any other map-local state read by the update helper—and set $\rho_{\mathrm{obs}}(\sigma) = R_{G0}(\sigma)$. The occupied-key set $K_{G0}(\sigma)$ is only a derived projection of $R_{G0}(\sigma)$. Inputs $(0, 1)$ and $(1, 1)$ reach states σ_0 and σ_1 at ℓ with $K_{G0}(\sigma_0) = \{S\}$ and $K_{G0}(\sigma_1) = \{S, A\}$, hence $R_{G0}(\sigma_0) \neq R_{G0}(\sigma_1)$.

Take the common remaining suffix w to be the fresh-B update followed by the uniform capture and store of its success bit, and define $\mathsf{Obs}(\tau) = [ret(\tau) = 0]$. The slice and ctx_w include the common program point, key B, value, $G0$ identity and static attributes (map type, capacity, default preallocation, and BPF_ANY mode), relevant control and wire values, and every suffix-read component outside R_{G0}. The contract's Env fixes the kernel, object, map instances, schedule, and absence of interference. Under the stated map-update contract, the two suffixes terminate with observations 1 and 0. The earlier input is not read by the suffix, and its persistent suffix-relevant effect is contained in the selected complete $G0$ dynamic state, so the non-selected contexts agree. Thus w is a conditional Definition 1 witness under this declared concrete-service contract. This is stronger than selecting occupancy alone and does not claim that the retained traces expose all internal kernel fields.

Under the gate discipline, take $s_{D_G}(\sigma) = (phase(\sigma), K(\sigma))$ on the invariant region X_{D_G} fixed by valid arguments, the dedicated preallocated map contract, successful reset, serialization, and no interference. Adequacy on this region is an explicit service-contract premise, not a machine-checked model of every kernel field. Phase ranges over the reset/sentinel/first/second-operation stages and $K \subseteq \{S, A, B\}$ with $|K| \leq 2$. The carrier, and hence its future-observation quotient, is finite. For E1, the two complete gate words for $(0, 1)$ and $(1, 1)$ have prefixes $updS$ and $insA$ and the common remaining suffix just defined. The complete reset-plus-gate word need not be causal because reset erases incoming differences; the witness is tagged by the internal frontier after the first operation.

For the basis take $P_G = P_U$, $\mathsf{Adm}_G = X_{D_G}$, reset as delete S, A, B then insert S, G as the $S/A/B$ selector, and $observe = [\text{second return} = 0]$. The witness above discharges E1; Proposition 2 and the reset premises discharge E2–E3.

This mechanism adds no extensional Boolean expressiveness to eBPF. It is a calibration case showing that a documented stateful service can supply a reusable post-acceptance gate; it does not by itself establish report omission.

C. Fixed interpreter and map ABI

In addition to $G0$, $wm\_circuit$ uses $TAPE$ and four interpreter maps:

- $TAPE$ records build variant, capacity, selected raw return, and error count;
- $CIRCUIT$ stores normalized records $(op, src0, src1, dst)$;
- $WIRES$ stores constants, primary inputs, and canonical SSA-style gate outputs;
- $VM\_CONTROL$ supplies the ABI version and declared counts;
- VM_TRACE stores per-gate validity, output, and, for state-mediated variants, the raw second-helper return.

The host parser checks and normalizes WMC1, writes map cells, and retains the requested output list. The list is

TABLE II
CARRIER BOUNDARY IN THE EBPF CALIBRATION CASE.

Host data and validation	Recognition unit	Post-acceptance interpretation
textual WMC1 $\rightarrow$ normalized map configuration $\mathsf{Enc}_U(d,x)$	V accepts the fixed P_U, not WMC1 or d	P_U reads maps $\rightarrow \mathsf{Sched}_U(d,x) \rightarrow$ helper returns and wire observations

TABLE III
SATURATING-RANK NAND EXECUTIONS.

a	b	Operation word after reset	State before second operation	Second result	Output $[ret = 0]$
0	0	$updS; updS$	$\{S\}$	success	1
0	1	$updS; insB$	$\{S\}$	success	1
1	0	$insA; updS$	$\{S, A\}$	success	1
1	1	$insA; insB$	$\{S, A\}$	failure	0

projected only after $status = \mathsf{OK}$ and is never read by BPF. The source-level mask is guarded by host control flow; the negative suite tests rejection status and execution counts, not an injected runtime masking path. Inside the kernel, a bounded loop handles at most 512 descriptors. Each iteration copies descriptor and source values before helper calls, resets $G0$, invokes the gate, and updates the canonical destination by key, avoiding retention of map-value pointers across those calls.

Canonical destinations prevent source/destination aliases and forward references. Covered malformed versions, counts, opcodes, destinations, or sources receive non-OK status. For valid descriptors, the topological restriction makes every source earlier. External whole-transaction serialization prevents concurrent mixed configurations; successive serial invocations overwrite shared cells, and stale physical cells may remain.

The verifier's acceptance unit is the fixed interpreter artifact rather than each descriptor, although the harness may reload the same object for different datasets. Changing map configuration changes the interpreted circuit. This is the E4-D carrier distinction, not evidence for a compiler that emits a new accepted BPF object per circuit.

D. Source-level invariant and evidence boundary

Lemma 1 (source-level interpreter prefix invariant). Assume the update laws of Proposition 2, successful reset and required map/helper calls, the E4-D serialization environment, valid $\mathsf{Enc}_U(d,x)$, and that the captured translated object preserves the manually inspected source control/data dependencies. For every integer t with $0 \le t \le n(d)$, after successful completion of the ordered callback indices $0, \ldots, t-1$, every canonical wire below $2 + m(d) + t$ equals the corresponding prefix of $\mathsf{Eval}_{\mathsf{NAND}}(d,x)$.

Proof sketch. The outer program initializes constants and preserves host-written inputs. A callback validates the current opcode, destination, and earlier sources, then copies the descriptor and source bits. Proposition 2 gives NAND after reset; success writes only the canonical destination and increments the completed count. The outer program requests $n(d)$ iterations through bpf_loop; under the helper contract, callback return zero continues, return one stops, and the helper reports the number of iterations performed [13]. The interpreter accepts only when both that return and its own completed count equal $n(d)$. Induction on t gives the prefix claim. This is a source-level argument supported by retained translated dumps for manual inspection, not a machine-checked eBPF-semantics proof. □

The preserved object, verifier metadata, and loaded-program tag support E4-D's fixed-acceptance unit. Source and translated dumps support initialization, ordered bpf_loop use, validation, reset, one-destination writes, and bounds; audits exercise outputs and negative status/count behavior. Serialization remains external because the program has no lock. Thus A is recorded, C has a conditional witness under the declared map-service/no-interference contract, and P is supported under additional source-to-object, frame, and serialization premises. No stock-Linux report-cell extractor or deployment policy is supplied, so this calibration does not establish R or W.

V. EVALUATION

A. Recorded environment and primary run

The primary evidence is the run bundle `results/interpreter/interpreter-final-20260711-02/`. It records Ubuntu 24.04, Linux 6.17.0-35-generic on aarch64, and preserves four BPF variants, captured loaded-program metadata, verifier logs, descriptors, JSONL outputs, an author-run semantic audit, a then-current selected source/manuscript snapshot, and a self-issued SHA-256 manifest. The final manuscript postdates that run and is not claimed to be covered by its manifest. The bundle is author-generated and separately author-audited, not a third-party reproduction.

The dataset contains exactly

$$\begin{aligned} 38{,}533 = {} & 26{,}488 \text{ per-gate records} \\ & + 12{,}037 \text{ successful run records} \\ & + 8 \text{ negative-control records.} \end{aligned} \tag{31}$$

These are heterogeneous evidence rows, not "38,533 tests." In particular, the explicit arithmetic baseline's gate records do not contain state-mediated helper-return traces.

TABLE IV
RECORDED EVALUATION COVERAGE.

Dataset and input coverage	Successful runs	Per-gate records
9 named circuits, exhaustive per circuit	39	166
100 random DAGs, exhaustive per DAG ($\leq$ 6 inputs, $\leq$ 24 gates; seed 3235823838)	1,876	23,776
512-gate deep and 64-input/512-gate/578-wire joint boundaries	4	2,048
zero-gate descriptor; 10,000 serial NAND/adder/mux invocations	10,001	0
capacity-64, forced-sentinel, and explicit-logic controls	117	498
8 malformed ABI/count/op/destination/reference cases	—	—

B. Coverage

Named and random circuits exhaust their own finite inputs; the joint 64-input boundary uses only all-zero and all-one vectors, not 2^{64} inputs. The corpus is regression evidence, not an enumeration of $D_{64,512}$. Serial reuse detects common reset/stale-state regressions but is neither a concurrent stress test nor proof of the external critical-section premise.

C. Separate semantic audit and integrity check

The author-run auditor does not rely solely on the runner's pass flag: it also reconstructs named and boundary descriptors, regenerates the seeded 100-DAG corpus, independently evaluates complete wire vectors, checks destinations, boundary gates and eight malformed cases, and matches runtime tags to loaded-program metadata. It reports success. A subsequently written self-issued SHA-256 manifest also verifies; it detects drift but is not a signature, timestamp, attestation, or independent reproduction.

D. Mechanism attribution

Three variants separate the capacity mechanism from the truth table and artifact acceptance.

First, increasing capacity from 2 to 64 removes saturation in the tested schedule; all 166 named-corpus control gate outputs become 1. Second, forcing the second one-bit input to update S removes the fresh-B insertion; again all 166 outputs become 1. Third, an arithmetic baseline computes NAND without the capacity mechanism and produces the expected named-circuit results. All four variants—the state-mediated gate, two controls, and baseline—load and execute in the recorded environment.

The controls support mechanism attribution: saturation and the second fresh name are both necessary for the gate's zero output. They do not show equivalent verifier reports. The baseline confirms that the mechanism gains no new Boolean function unavailable to ordinary bytecode.

E. Calibration result

Linux's verifier documentation states its objective in terms of determining program safety and validating paths, arguments, and memory access [14]; it does not specify a complete functional certificate for map-mediated computation. Without a stock-Linux report extractor we neither test nor infer how precisely it tracks this helper return. The case gives a conditional C witness under the declared service contract and supports P under additional premises; it is not evidence of unsound acceptance, and R remains unestablished for this stock-Linux calibration.

F. Fixed auxiliary executable report instance

To exercise the report-relative criterion without relabeling Linux verifier artifacts, we embed the auxiliary R instance in the full model carrier of Section 7.1 and fix

$$M_{linux\_r\_aux\_v1} = (V_{linux\_r}, I_{hash}, \mathsf{Report}_{aux}, K_{\mathrm{obs}}, P_{aux}, \ell, D, F, \varnothing, \varnothing, \varnothing, \varnothing). \tag{32}$$

P_{aux} is accepted by the custom report-producing recognizer V_{linux_r}; it is not a BPF object accepted by the stock Linux verifier. I_{hash} is a finite, deterministic, serialized, no-interference semantics restricted to the non-evicting HASH-map update cases in the fixed program. Here Report_{aux} means only `report.json["report_cells"]`; `derivation.json` records worklist/computation provenance and is not part of the report-label interface. The last four components are the typed empty actor set, effect set, drive relation, and permitted-effect set; they embed the R instance in the common carrier and make no W claim.

Executable certificate. $R(M_{linux\_r\_aux\_v1})$ holds (artifact status: established).

Certificate argument. The recognizer accepts P_{aux}. Exhausting $a \in \{0,1\}$ with $b = 1$ in the singleton serialized/no-interference environment gives $F = \mathsf{Reach}_{I_{hash}}(P_{aux}, \ell) = \{\texttt{frontier:S}, \texttt{frontier:AS}\} \subseteq X_D$. Here X_D additionally contains the corresponding two terminal states. Let $a_{\mathrm{obs}} \in A_D$ be the discipline action named `update-suffix-and-observe`, write c_{obs} for the exact encoded runtime-operation symbol `bpf_map_update_elem(G0,suffix_key,one,BPF_ANY);observe(ret==0)`, and set $\iota_{P_{aux},\ell}(a_{\mathrm{obs}}) = c_{\mathrm{obs}}$; this singleton encoding is injective. Concrete execution of $\iota_{P_{aux},\ell}(a_{\mathrm{obs}})$ is defined on the two frontier states, undefined on the terminal states, and agrees with D on definedness, output, and successor. Thus the checker establishes operational adequacy on all of X_D. The observation contract fixes ρ_{obs} as the occupied-key set, Obs as the ordered success-bit word, Slice as the program-phase/service-context pair, and Env as that singleton environment; in particular, $\rho_{\mathrm{obs}}(\texttt{frontier:S}) = \{S\} \neq \{S, A\} = \rho_{\mathrm{obs}}(\texttt{frontier:AS})$. For every ordered pair in F^2 and every word in $\mathcal{W}_D(F) = \{\varepsilon, a_{\mathrm{obs}}\}$, the checker establishes the remaining runtime-word, common-context, observer-compatibility, and K_{obs}-soundness obligations. These discharge every admissibility item except unique-cell coverage.

Report_{aux} emits one unique cell $a^{\#}$ that covers both frontier states, completing $\mathsf{Adm}(P_{aux}, \ell, D, F; K_{\mathrm{obs}})$. The exact finite future-observation quotient assigns them different classes, so $|\beta_D(F_{a^{\#}})| = 2$; moreover a_{obs} yields observations 1 and 0. Hence the same suffix first witnesses Definition 1 and the tagged tuple $(P_{aux}, \ell, D, F, a^{\#}, a_{\mathrm{obs}})$ satisfies Definition 2. Therefore $R(M_{linux\_r\_aux\_v1})$ holds. The evidence also records 21 domain/action *return-class containment* checks plus two report-cell *successor-containment* checks, all with zero violations; the former are not 21 full post-state checks. Each of the four negative controls—exact occupancy tracking, capacity 64, forced sentinel, and an unobserved return—removes the R witness. □

An independent executable checker reconstructs reachability, unique-cell coverage, the quotient, and the factorization verdict without importing the model implementation. The archived VM run and audit passed; the unit suite constructs two fixed-input bundles and byte-compares their formal JSON. This is executable finite-model evidence, not a machine-checked proof. The retained kernel calibration has four oracle rows for the two assignments $(0, 1)$ and $(1, 1)$; it calibrates the restricted service cases only. It proves no refinement or bisimulation between I_{hash} and Linux, and extracts no stock-verifier cell. Thus R for stock Linux, W, $\mathsf{WM}_{\mathrm{shape}}$, and any universal necessity conjecture remain unestablished. The bundle is `results/linux_r/linux-r-v1/`.

G. Threats to validity

The experiment uses one kernel build and architecture. The argument relies on a dedicated preallocated non-LRU map, successful reset, distinct keys, the stated update laws, and whole-map-set mutual exclusion. Gate-emitting datasets preserve raw returns; the 10,000-run stress file records run/status/output evidence without per-gate raw rows. The proof uses only success versus failure and promises no portable error number.

Finite testing does not prove all $D_{64,512}$. Theorem 1 instead depends on the stated source-level initialization, validation, ordered iteration, gate, and frame obligations; the corpus seeks counterexamples. Translated dumps and verifier logs are retained for manual inspection and manifest hashing, not consumed by an automated data-flow checker. Status masking has a source-order guard, while the negative suite tests rejection/status rather than an injected masking path. There is no machine-checked eBPF-semantics proof, and production-verifier safety soundness is assumed only for the optional Safe conclusion.

VI. Related Work

Language-theoretic security supplies the recognition/interpretation framing [1]–[4]; object-centric tracing reconstructs state-dependent low-level operation languages [15]. Weird-machine work studies unintended computation and exploitability [16], [17], insecure compilation makes the policy boundary explicit [18], and proof-carrying-code work identifies shadow execution outside a proof model [19]. Valid-input computation also appears in well-formed RPM metadata, while vulnerability-flow types derive abstract weird machines [20], [21]; neither adopts our actual-report-cell criterion for one accepted artifact.

Abstract interpretation supplies concrete/abstract, soundness, and completeness vocabulary [5], [22]; our criterion is limited to uniquely assigned cells actually computed at one frontier. MOAT isolates post-acceptance BPF effects [23], while mismorphism compares interpretations [24]. For eBPF, range-analysis verification and state embedding target verifier-logic soundness/coverage errors [25], [26]. VEP's "programmability" means reducing verification-toolchain restrictions, while Rex addresses rejection-side language–verifier mismatch by replacing a separate static verifier with language-based safety and an extralingual runtime [27], [28]; P here instead means one fixed stock-verifier-accepted artifact interpreting a bounded family. DRACO checks functional specifications after verifier acceptance, and BPFVERIFY translates eBPF bytecode to bit- and memory-precise Horn clauses for functional verification [29], [30].

Recent preprints further separate the boundary: Heimdall treats higher-level defects surviving compilation and acceptance; Yaksha-Prashna extracts third-party bytecode behavior; bpfix localizes proof loss in rejected programs [31]–[33]. Trust-boundary semantic-gap work studies insufficient assertions after correctly implemented syntactic acceptance [34]. They do not jointly distinguish the obligations in our claim graph; conversely, our stock-Linux calibration case does not establish R or W, while the separate auxiliary tuple establishes only its own R. These four 2026 works are cited as preprints, not peer-reviewed publications.

VII. Limitations, Implications, and Outlook

A. The claim graph has strict separations

To put all five nodes on one carrier, fix a model

$$M = (V, I, \mathsf{Report}, K_{\mathrm{obs}}, P_*, \ell_*, D_R, F_R, \mathcal{A}, \mathcal{E}, \mathsf{Drive}, \mathsf{Pol}), \tag{33}$$

enriched with named intended-semantics, safety, report-abstraction, and granularity contracts against which the evidence predicates below are evaluated, where $\mathcal{A}$ and $\mathcal{E}$ are actor and effect sets, $\mathsf{Drive} \subseteq \mathcal{A} \times \mathcal{E}$ records effects actors can drive through the designated P_*, and $\mathsf{Pol} \subseteq \mathcal{E}$ is the permitted-effect set. Define $A(M)$ by $P_* \in L_V$; $C(M)$ by $\exists \ell, \exists w.\, (P_*, \ell, w) \in L_{\mathrm{causal}}(V, I; K_{\mathrm{obs}})$; $P(M)$ when P_* is one fixed accepted bounded interpreter with a same-artifact basis $(P_*, reset, G, observe, D_G, \mathsf{Adm}_G)$ satisfying E1–E3, P_* discharges E4-D using that basis, and g together with constants $\{0, 1\}$ is functionally complete for Boolean circuits; $R(M)$ when $\mathsf{Adm}(P_*, \ell_*, D_R, F_R; K_{\mathrm{obs}})$ holds and some $(P_*, \ell_*, D_R, F_R, a^{\#}, w)$ is output-witnessed report-relative residual on exactly that tuple; and $W(M)$ when $\exists a \in \mathcal{A}, \exists e \in \mathcal{E}.\, (a, e) \in \mathsf{Drive} \wedge e \notin \mathsf{Pol}$.

For classification, let $\mathsf{Link}(M)$ require the R witness (when used) to lie in the gate/interpreter execution family establishing P and the W effect to arise by an actor driving that same

encoded computation; unrelated co-resident witnesses do not qualify. Let $\mathsf{Unint}(M)$ mean that this programmable interpretation/effect is outside the boundary's intended semantics. Define

$$\mathsf{WM}_{\mathrm{policy}}(M) = P(M) \wedge W(M) \wedge \mathsf{Link}(M) \wedge \mathsf{Unint}(M). \tag{34}$$

Let $\mathsf{Doc}(M)$ mean that this linked behavior uses documented semantics and preserves the declared safety contract, $\mathsf{Conf}(M)$ mean that the report implementation conforms to its specified abstraction, and $\mathsf{Gran}(M)$ mean that its R collision is forced by the declared report granularity. Define the narrower class by

$$\begin{aligned}\mathsf{WM}_{\mathrm{shape}}(M) = \mathsf{WM}_{\mathrm{policy}}(M) \wedge R(M) \\ \wedge \mathsf{Doc}(M) \wedge \mathsf{Conf}(M) \wedge \mathsf{Gran}(M).\end{aligned} \tag{35}$$

Proposition 3 (non-implications among claim nodes). Over this model class, the following implications are invalid:

$$\begin{gathered}A \not\Rightarrow C, \quad C \not\Rightarrow P, \quad C \not\Rightarrow R, \quad P \not\Rightarrow R, \\ R \not\Rightarrow P, \quad P \not\Rightarrow W, \quad R \not\Rightarrow W.\end{gathered} \tag{36}$$

The definitions validate $R \Rightarrow C \Rightarrow A$ and $P \Rightarrow C \Rightarrow A$: Definition 2 contains a Definition 1 witness, and E1 uses the E4-D artifact.

Proof by finite countermodels. For $A \not\Rightarrow C$, accept only *skip*. For $C \not\Rightarrow P$ and $R \not\Rightarrow P$, use states $\{0, 1, \perp\}$, a one-shot bit read, and one report cell. Exact future-class cells give $C \not\Rightarrow R$; an E1–E4-D NAND interpreter with that exact partition gives $P \not\Rightarrow R$. Setting $\mathsf{Pol} = \mathcal{E}$ gives $P \not\Rightarrow W$ and, in the one-shot model, $R \not\Rightarrow W$. Finally, an actor-driven, unintended, policy-excluded effect from the same encoded P family with an exact report partition gives $\mathsf{WM}_{\mathrm{policy}} \wedge \neg R$. □

Thus bare C, abstract-interpretation incompleteness, or acceptance incompleteness does not imply P or a weird-machine classification.

B. Defensive implications

Audit the recognized property and report separately, enumerate actor-driven operations and environment assumptions, then test report factorization. Contract violations call for implementation fixes; documented collisions call for report refinement, restriction, or isolation only when the report intended that relation.

C. Weird-machine status and a future shape theorem

The stock eBPF case supports P under its source/object and serialization premises but establishes neither R nor W. The auxiliary $M_{linux\_r\_aux\_v1}$ establishes R only for its fixed custom report and restricted service semantics; it establishes neither W nor $\mathsf{WM}_{\mathrm{shape}}$ and does not discharge the missing stock-report embedding. A shape theorem must derive **macro closure** of acceptance and **report embedding** of the collision; context sensitivity can defeat either. Complete-shell theory [22] may characterize report refinement, but cannot create reachability, reset, routing, linkage, or policy violation.

VIII. Conclusion

Artifact acceptance and post-acceptance interpretation use different languages. We separate causal words from output-witnessed report residuals and give factorization and bounded-composition criteria. The stock eBPF case records A, conditionally witnesses C, and supports P for a fixed 64-input/512-gate NAND interpreter, but establishes neither R nor W. A separate fixed auxiliary tuple establishes R by an actual computed-cell collision, without establishing W, $\mathsf{WM}_{\mathrm{shape}}$, a stock-Linux report result, or a universal necessity theorem.

Ethics and Data Availability

Experiments run in an isolated local VM, use no live hook or third-party target, and attempt no corruption, verifier bypass, or privilege escalation.

The repository is https://github.com/Emtanling/eBPF-machine; immutable eBPF-interpreter evidence is at commit `4309069a`, under `results/interpreter/interpreter-final-20260711-02/`. The auxiliary report-instance evidence is at commit `f665b1a`, under `results/linux_r/linux-r-v1/`. The bundles contain their relevant model or program inputs, outputs, audit material, and self-issued integrity manifests; those manifests protect against accidental drift, not independent provenance.

Acknowledgment and AI-Assistance Disclosure

OpenAI Codex assisted drafting, revision, language editing, artifact code/test revision, and author-directed checks. The author independently reviewed the claims, proofs, citations, changes, and results and takes full responsibility. No conflict of interest is declared.

References

[1] L. Sassaman, M. L. Patterson, S. Bratus, and M. E. Locasto, "Security Applications of Formal Language Theory," *IEEE Systems Journal*, vol. 7, no. 3, pp. 489–500, 2013, doi: 10.1109/JSYST.2012.2222000.

[2] F. Momot, S. Bratus, S. M. Hallberg, and M. L. Patterson, "The Seven Turrets of Babel: A Taxonomy of LangSec Errors and How to Expunge Them," in *2016 IEEE Cybersecurity Development (SecDev)*, pp. 45–52, 2016, doi: 10.1109/SecDev.2016.019.

[3] L. Sassaman, M. L. Patterson, and S. Bratus, "A Patch for Postel's Robustness Principle," *IEEE Security & Privacy*, vol. 10, no. 2, pp. 87–91, 2012, doi: 10.1109/MSP.2012.31.

[4] S. Ali, P. Anantharaman, Z. Lucas, and S. W. Smith, "What We Have Here Is Failure to Validate: Summer of LangSec," *IEEE Security & Privacy*, vol. 19, no. 3, pp. 17–23, 2021, doi: 10.1109/MSEC.2021.3059167.

[5] P. Cousot and R. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '77)*, pp. 238–252, 1977, doi: 10.1145/512950.512973.

[6] A. Nerode, "Linear Automaton Transformations," *Proceedings of the American Mathematical Society*, vol. 9, no. 4, pp. 541–544, 1958, doi: 10.1090/S0002-9939-1958-0135681-9.

[7] G. H. Mealy, "A Method for Synthesizing Sequential Circuits," *Bell System Technical Journal*, vol. 34, no. 5, pp. 1045–1079, 1955, doi: 10.1002/j.1538-7305.1955.tb03788.x.

[8] G. Amato and F. Scozzari, "Observational Completeness on Abstract Interpretation," *Fundamenta Informaticae*, vol. 106, nos. 2–4, pp. 149–173, 2011, doi: 10.3233/FI-2011-381.

[9] F. Ranzato and F. Tapparo, "Generalized Strong Preservation by Abstract Interpretation," *Journal of Logic and Computation*, vol. 17, no. 1, pp. 157–197, 2007, doi: 10.1093/logcom/exl035.

[10] Linux Kernel Documentation, "Running BPF Programs from Userspace," [Online]. Available: Linux v6.17 BPF documentation (accessed Jul. 11, 2026).

[11] Linux Kernel Documentation, "Program Types and ELF Sections," [Online]. Available: Linux v6.17 libbpf documentation (accessed Jul. 11, 2026).

[12] Linux Kernel Documentation, "BPF_MAP_TYPE_HASH, with PER-CPU and LRU Variants," [Online]. Available: Linux v6.17 map documentation (accessed Jul. 11, 2026).

[13] Linux Kernel Developers, "bpf_loop Helper API," *include/uapi/linux/bpf.h*, Linux v6.17. [Online]. Available: Linux v6.17 source (accessed Jul. 13, 2026).

[14] Linux Kernel Documentation, "eBPF Verifier," [Online]. Available: Linux v6.17 verifier documentation (accessed Jul. 11, 2026).

[15] I. Palmer, E. Rogers, and R. Adams, "Object-Centric Tracing for Language-Theoretic Security in Low-Level Interfaces," in *Twelfth Workshop on Language-Theoretic Security (LangSec 2026)*, 2026. [Online]. Available: langsec.org (accessed Jul. 12, 2026).

[16] S. Bratus, M. E. Locasto, M. L. Patterson, L. Sassaman, and A. Shubina, "Exploit Programming: From Buffer Overflows to Weird Machines and Theory of Computation," *USENIX ;login:*, vol. 36, no. 6, pp. 13–21, 2011.

[17] T. Dullien, "Weird Machines, Exploitability, and Provable Unexploitability," *IEEE Transactions on Emerging Topics in Computing*, vol. 8, no. 2, pp. 391–403, 2020, doi: 10.1109/TETC.2017.2785299.

[18] J. Paykin et al., "Weird Machines as Insecure Compilation," arXiv:1911.00157, 2019, doi: 10.48550/arXiv.1911.00157.

[19] J. Vanegue, "The Weird Machines in Proof-Carrying Code," in *2014 IEEE Security and Privacy Workshops*, pp. 209–213, 2014, doi: 10.1109/SPW.2014.37.

[20] S. Ali, M. E. Locasto, and S. W. Smith, "Weird Machines in Package Managers: A Case Study of Input Language Complexity and Emergent Execution in Software Systems," in *2024 IEEE Security and Privacy Workshops (SPW)*, pp. 169–179, 2024, doi: 10.1109/SPW63631.2024.00021.

[21] M. Lesani, "Vulnerability Flow Type Systems," in *2024 IEEE Security and Privacy Workshops (SPW)*, pp. 157–168, 2024, doi: 10.1109/SPW63631.2024.00020.

[22] R. Giacobazzi, F. Ranzato, and F. Scozzari, "Making Abstract Interpretations Complete," *Journal of the ACM*, vol. 47, no. 2, pp. 361–416, 2000, doi: 10.1145/333979.333989.

[23] H. Lu, S. Wang, Y. Wu, W. He, and F. Zhang, "MOAT: Towards Safe BPF Kernel Extension," in *33rd USENIX Security Symposium*, pp. 1153–1170, 2024.

[24] P. Anantharaman et al., "Mismorphism: The Heart of the Weird Machine," in *Security Protocols XXVII*, LNCS 12287, pp. 113–124, 2020, doi: 10.1007/978-3-030-57043-9_11.

[25] H. Vishwanathan, M. Shachnai, S. Narayana, and S. Nagarakatte, "Verifying the Verifier: eBPF Range Analysis Verification," in *Computer Aided Verification*, Lecture Notes in Computer Science, vol. 13966, pp. 226–251, 2023, doi: 10.1007/978-3-031-37709-9_12.

[26] H. Sun and Z. Su, "Validating the eBPF Verifier via State Embedding," in *18th USENIX Symposium on Operating Systems Design and Implementation*, pp. 615–628, 2024.

[27] X. Wu et al., "VEP: A Two-stage Verification Toolchain for Full eBPF Programmability," in *22nd USENIX Symposium on Networked Systems Design and Implementation*, pp. 277–299, 2025.

[28] J. Jia et al., "Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions," in *2025 USENIX Annual Technical Conference*, pp. 325–342, 2025.

[29] D. Lu, B. Tang, M. Paper, and M. Kogias, "Towards Functional Verification of eBPF Programs," in *Proceedings of the 2024 ACM SIGCOMM Workshop on eBPF and Kernel Extensions (eBPF '24)*, pp. 37–43, 2024, doi: 10.1145/3672197.3673435.

[30] M. Bromberger, S. Schwarz, and C. Weidenbach, "Automatic Bit- and Memory-Precise Verification of eBPF Code," in *Proceedings of the 25th Conference on Logic for Programming, Artificial Intelligence and Reasoning*, EPiC Series in Computing, vol. 100, pp. 198–221, 2024, doi: 10.29007/sj4l.

[31] V. A. Dasu, M. Santra, M. R. U. Rashid, A. Kumar, S. Tizpaz-Niari, and G. Tan, "Heimdall: Formally Verified Automated Migration of Legacy eBPF Programs to Rust," arXiv:2605.25411, 2026, doi: 10.48550/arXiv.2605.25411.

[32] A. Singh et al., "Yaksha-Prashna: Understanding eBPF Bytecode Network Function Behavior," arXiv:2602.11232, 2026, doi: 10.48550/arXiv.2602.11232.

[33] Y. Zheng et al., "Characterizing and Bridging the Diagnostic Gap in eBPF Verifier Rejections," arXiv:2607.02748, 2026, doi: 10.48550/arXiv.2607.02748.

[34] D. Kim, J.-Y. Choi, and J. Lee, "Trust Boundary Semantic Gaps: A Multi-dimensional Analysis and Mitigation for Security-by-Design," arXiv:2607.01711, 2026, doi: 10.48550/arXiv.2607.01711.